Showtime Hub

Software Requirements Specification

Version 1

09/18/2026

Group #9
Ivan Rodriguez, Sang Duong, Diyor Sattarov

Prepared for
CS 250- Introduction to Software Systems
Instructor: Gus Hanna, Ph.D.
Fall 2026

<Showtime Hub>

Revision History

Date	Description	Author	Comments
<date>	<Version 1>	<Your Name>	<First Revision>
09/24/2026	Version 1	Team	First draft
10/8/2026	Version 2	Team	SRS with Design Portion
10/22/2026	Version 3	Team	SRS with Test Cases

Document Approval

The following Software Requirements Specification has been accepted and approved by the following:

Signature	Printed Name	Title	Date
	<Your Name>	Software Eng.	
	Dr. Gus Hanna	Instructor, CS 250	

Table of Contents

REVISION HISTORY 2

DOCUMENT APPROVAL 3

1. INTRODUCTION 4

1.1 PURPOSE 5
1.2 SCOPE 5
1.3 DEFINITIONS, ACRONYMS, AND ABBREVIATIONS 5
1.4 REFERENCES 7
1.5 OVERVIEW 7

2. GENERAL DESCRIPTION 7

2.1 PRODUCT PERSPECTIVE 7
2.2 PRODUCT FUNCTIONS 7
2.3 USER CHARACTERISTICS 8
2.4 GENERAL CONSTRAINTS 8
2.5 ASSUMPTIONS AND DEPENDENCIES 8

3. SPECIFIC REQUIREMENTS 8

3.1 EXTERNAL INTERFACE REQUIREMENTS 8
3.1.1 User Interfaces 8
3.1.2 Hardware Interfaces 9
3.1.3 Software Interfaces 9
3.1.4 Communications Interfaces 9
3.2 FUNCTIONAL REQUIREMENTS 9
3.2.1 Ticket Purchasing 9
3.2.2 Concession (Food and Drinks) Purchase 10
3.3 USE CASES 11
3.3.1 Purchase Movie Tickets 11
3.3.2 Purchase Concessions 12
3.3.3: Browse Movie Information 14
3.4 CLASSES / OBJECTS 16
3.4.1 Class Customer 16
3.4.2 Class Payment Method 17
3.5 NON-FUNCTIONAL REQUIREMENTS 17
3.5.1 Performance 17
3.5.2 Reliability 17
3.5.3 Availability 17
3.5.4 Security 17
3.5.5 Maintainability 17
3.5.6 Portability 17

4. SOFTWARE DESIGN SPECIFICATION 18

4.1 SYSTEM DESCRIPTION 18
4.2 SOFTWARE ARCHITECTURE OVERVIEW 18
4.2.1 Architectural Diagram 18
4.2.2 UML Class Diagram 19
4.3 CLASS DESCRIPTIONS 20
4.3.1 Customer 20
4.3.2 PaymentMethod 21
4.3.3 Movie 22
4.3.4 Theater 22
4.3.5 Showtime 23

4.3.6 Seat 24
4.3.7 Ticket 24
4.3.8 ConcessionOrder 25
4.3.9 Payment 26
4.4 DEVELOPMENT PLAN AND TIMELINE 26
4.4.1 Team Responsibilities 27
4.4.2 Development Timeline 27

5. TEST PLAN 27

5.1 PURPOSE AND SCOPE 28
5.3 TESTING METHODS 28
5.3.1 Unit Testing 29
5.3.2 Functional Tests 29
5.4 TEST CASES AND CONNECTIONS TO SRS 31
5.5 ERRORS AND LIMITS OF THE TESTS 31
5.6 RECORDING TEST RESULTS 32
5.7 TEST DOCUMENTS AND GITHUB LINKS 32

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) defines the requirements for ShowTime Hub, an online ticketing and concession-ordering website for participating movie theaters. It establishes the functionality and operational expectations the system must meet to support customers, theater staff, and administrators.

This document is intended for software developers, designers, testers, project managers, and theater management. It provides a shared reference for development decisions, system testing, and acceptance of the completed product.

1.2 Scope

ShowTime Hub will be a movie ticketing and concession-ordering platform accessible through a website. Customers can browse current and upcoming movies, view movie details and reviews, and search for screenings by title, date, time, location, and viewing format. Listings will include special events and information about available services.

Customers will be able to view seat availability, select seats, purchase individual or group tickets, and receive electronic tickets with a unique identifier. They can also purchase concessions and receive a claim code for pickup. Customer accounts will support purchase history, saved payment methods, rewards, badges, notifications, and sharing showtime and seating information.

Additional features will include movie trailers, user reviews and ratings, available critic scores, and recommendations for similar or trending films. Customers can coordinate group ticket purchases and view accessibility information, including closed captioning and audio description. System administrators will manage platform-wide settings and season themes.

The goal of ShowTime Hub is to bring movie discovery, visit planning, and purchasing into one platform, reduce the need for box-office purchases, and give theater staff a central system for managing screening information and online sales.

1.3 Definitions, Acronyms, and Abbreviations

Term	Definition
SRS	Software Requirements Specification: A document describing the system's required features and constraints.

API	Application Programming Interface: Allows ShowTime Hub to exchange information with other systems.
Showtime	A scheduled movie screening at a specific theater, date, and time.
Viewing Format	The presentation format of a screening, such as standard, 3D, 4DX, or IMAX.
Seat map	A visual layout showing seats, availability, and designated accessible seating.
E-Ticket	An electronic admission ticket with a unique identifier, viewable on a device or printable.
Concessions	Food and beverages available for purchase at a theater.
Claim code	A unique code used to identify a concession order during pickup.
Reward points	Credits earned through eligible purchases and redeemed according to program rules.
Badge	A digital achievement awarded for meeting specified movie-attendance criteria.
Closed captioning	Text displaying dialogue and relevant sounds through a supported device or display.
Audio Description	Spoken descriptions of important visual elements in a movie.
SMS	Short Message Service: text messaging used for customer notifications.
Theater Administrator	An authorized user who manages information and operations for assigned theater locations.
System Administrator	An authorized user who manages platform-wide settings, permissions, and content.
Rotten Tomatoes	Rotten Tomatoes is a popular review-aggregation website that rates movies and TV shows based on critic and audience reviews.
Screen X	A panoramic viewing format that extends selected movie imagery onto screens along the sides of the auditorium.
3D	A 3D movie is a motion picture that creates the illusion of depth so that objects on screen appear to exist in three-dimensional space when viewed through special glasses.
4DX	A 4DX movie is a “four-dimensional” cinema format that adds motion-synchronized seats and environmental effects—such as wind, water, fog, scents, and strobe lights—to enhance the on-screen action.
Trailer	A short promotional video previewing a movie.

IMAX	IMAX is a proprietary large-format cinema system using high-resolution cameras, specialized 70 mm film or advanced laser projection, and massive tall screens.
User Review	A movie user review is a personal evaluation of a film written by a viewer who has watched it, sharing their opinion, experience, and often detailed commentary, which can influence other audiences and even a movie's public perception
Critic Score	A movie evaluation provided by professional reviewers or an external review source.
Streaming Availability	Information identifying external services where a movie can be watched online.

1.4 References

No external documents are incorporated by reference in this version of the ShowTime Hub SRS. As business policies and service providers are completed, this section will include applicable theater policies, payment-service documentation, movie-info sources, and notification-service documentation.

Each reference will identify the document's title, author or publishing organization, date or version where available, and the location from which it can be obtained.

1.5 Overview

Section 2 will provide a general description of ShowTime Hub, including its relationship to theater operations, major functions, user characteristics, constraints, and assumptions and dependencies.

Section 3 defines the specific requirements for the website. It covers external interfaces. This includes movie discovery, ticket purchasing, concession ordering, and account management. It also covers use cases, classes, and objects. It also establishes requirements for performance, reliability, availability, security, maintainability, and portability.

2. General Description

2.1 Product Perspective

Showtime Hub is a web based movie ticketing and concession ordering system that provides an online alternative to purchasing tickets at a theater box office. It can be compared to systems like Fandango and Regal's online ticketing platforms. Customers can browse for movies and showtimes, view seat availability, make purchases online, and receive tickets on their phone that can be printed or displayed to the participating theater.

2.2 Product Functions

ShowTime Hub will include functions that allow the customers to browse movies that are currently showing or ones that are upcoming. Customers can filter the movie viewing format to (standard, 3D, 4DX, IMAX), select preferred showtimes, select seats from a visual, real-time seating display which also highlights sections for wheelchair accessibility and companions. Tickets can be purchased individually or as a group. After completing a secure credit or debit card payment, the customer will receive a unique ticket code and an electronic ticket that can be printed or displayed on a mobile device. The product will also allow customers to view past purchases, opportunities to leave a review, and a loyalty/rewards program in which points are earned through eligible purchases (movie tickets, food, and drinks). Authorized staff of participating theaters can update ticket prices, showtimes, and check sales reports. Product also allows participating theater companies to list their own movies, showtimes, and seat mappings on ShowTime Hub via an API.

2.3 User Characteristics

Users include the general customers with varying levels of technical experience, including customers who need simple interfaces; customers with needs such as the option for larger text, screen reader, and seat options. Administration needs access to pricing, movie scheduling, and can read sales reports.

2.4 General Constraints

Constraints would include a system that is usable on desktop and mobile web browsers; processing payments will be handled through a secure third-party payment service to reduce risk of fraud and protect customers. The system will be available throughout participating theaters' operating hours, except for scheduled maintenance for system updates.

2.5 Assumptions and Dependencies

Showtime Hub assumes that participating theaters will provide accurate movie, showtime, pricing, seating, and concession data through supported integrations. The system depends on reliable internet access, participating theater systems, and a third-party payment processor. Loyalty and rewards will be managed by authorized admins.

3. Specific Requirements

3.1 External Interface Requirements

3.1.1 User Interfaces

The website will have a web-based interface that works on both desktop and mobile devices. Users will be able to browse movies, check showtimes, choose seats, buy tickets, and order food. The interface that users can interact with will use clear navigation and readable/accessible text, so users can easily move through the different sections of the website.

3.1.2 Hardware Interfaces

The website does not need any special hardware to be accessed. Users can browse using a laptop, desktop computer, tablet, or mobile device with a valid internet connection.

3.1.3 Software Interfaces

The system will connect to participating theater systems to send and receive information like movies, showtimes, ticket prices, and availability. The website will also use an external service for receiving payments. The exact services and theater system APIs have not yet been decided.

3.1.4 Communications Interfaces

The system will use an internet connection to communicate with users, participating theaters, and other outside services. All information related to users, purchases, and theaters will be transferred through secure connections.

3.2 Functional Requirements

3.2.1 Ticket Purchasing

3.2.1.1 Introduction
The system will allow users to purchase tickets for available movie showtimes.

3.2.1.2 Inputs
The user will select a movie then theater, then showtime, then ticket quantity, and then available seat or seats. The user will also provide or select an already existing payment method to purchase the tickets.

3.2.1.3 Processing
The system will check any selected seats to verify availability and then calculate the total purchase cost. The payment will be sent to a payment service (like Stripe) for approval. Once processed, the order will be marked complete, and the seats will be marked unavailable.

3.2.1.4 Outputs
The system will display a purchase confirmation and generate a unique ticket ID or code for each purchased ticket.

3.2.1.5 Error Handling
If a payment is declined, the system will let the user know and the purchase will not be completed. The seat selection is saved if the user wants to retry. If a selected seat is no longer available, then the user will be asked to choose another seat.

3.2.2 Concession (Food and Drinks) Purchase

3.2.2.1 Introduction
The system will allow users to purchase available food and drink items from participating theaters.

3.2.2.2 Inputs
The user will select a theater location, then available concession items, then quantities, and a payment method.

3.2.2.3 Processing
The system will calculate the total cost of the order and send the payment for approval. Once approved, the order will be confirmed and recorded.

3.2.2.4 Outputs
The system will display an order confirmation and generate a unique claim code that can be used to pick up the order at the selected theater location.

3.2.2.5 Error Handling
If the payment is unsuccessful, the order will not be completed. Items are saved in the cart to be retried with different payment methods. If the item is out of stock, the user will be notified.

3.2.3 Browse Current and Upcoming Movies

3.2.3.1 Introduction

The system will allow users to browse current and upcoming movies and view their details without having to sign in.

3.2.3.2 Inputs

Customers will be able to select “Now Showing” or “Coming Soon”. Customers can also select nearby theater locations (by entering a ZIP code) and a movie to view.

3.2.3.3 Processing

The system will retrieve movie listings for the selected category and theater. When a movie is chosen, the system will retrieve its details and available showtimes.

3.2.3.4 Outputs

<Showtime Hub>

The system displays matching movies with their titles and posters. The selected movie page displays its Rotten Tomatoes score, runtime, synopsis, genre, director, cast, and available showtimes, along with trailers and review scores when available.

3.2.3.5 Error Handling

If no listings match, the system will display “No movies match your selected search criteria”. If the information cannot be loaded, the system displays an error message and allows the customer to retry.
…

3.3 Use Cases

3.3.1 Purchase Movie Tickets

Purpose: This use case explains how a customer purchases movie tickets through Showtime Hub.

Actors:
Customer (guest or signed in), external payment service, and participating theater system.

Preconditions:
Working internet connection, participating theater has the movie and showtime available for booking, and customers do not need an account to purchase tickets.

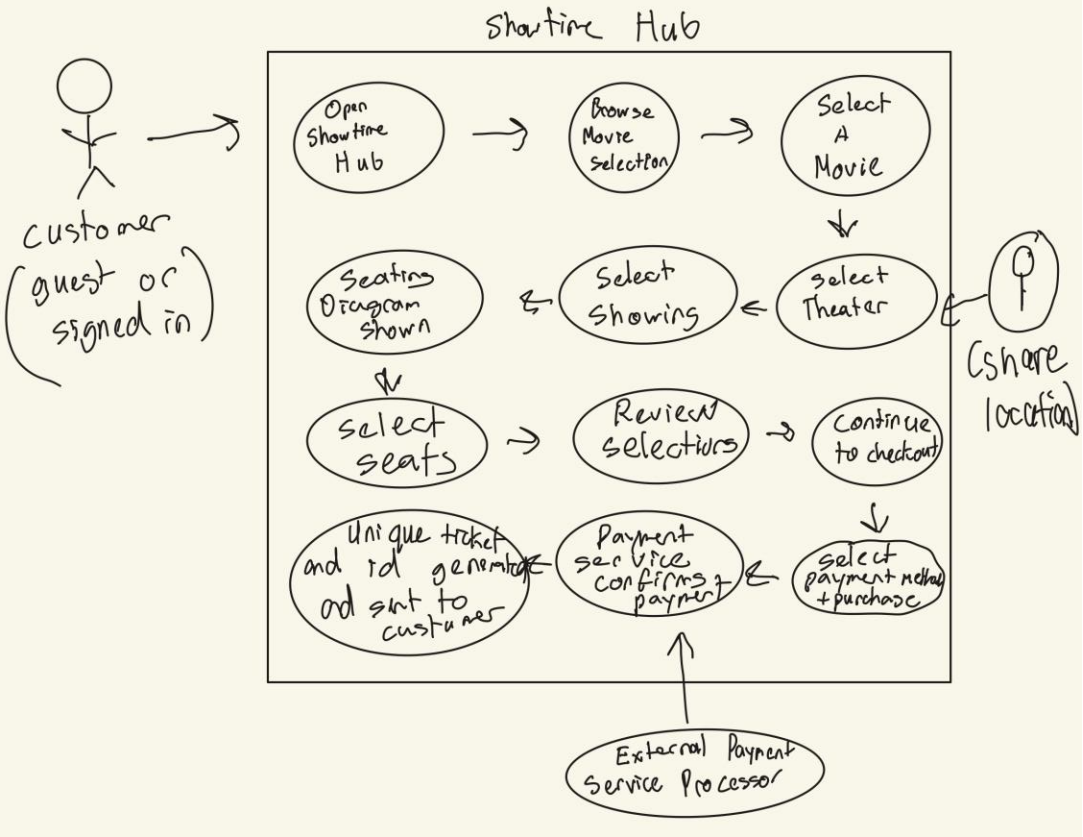
Events:

1. Customer opens Showtime Hub and browses the available movies
2. The customer selects a movie.
3. The customer enters or shares a location to view nearby theaters and their showtimes.
4. Once a showtime is picked, the system displays a seating diagram showing available and unavailable seats.
5. The system checks seat availability and displays the selected movie, theater, showtime, seats, and total price.
6. The customer reviews their selections and continues to checkout.
7. The customer enters their payment information. Signed in customers can also select a saved payment method
8. The customer confirms the purchase, and the system sends the payment request to the external payment service.
9. Once the payment is successful and the theater confirms the booking, the system marks the selected seats as sold.
10. The system generates an electronic ticket with a unique ID for each purchased seat and displays the tickets and purchase confirmation to the customer.

Events with Errors:

- If a selected seat is no longer available, the system asks the customer to choose another seat.
- If payment is declined, the system informs the customer and allows another payment attempt after checking seat availability again.
- If payment succeeds but the theater cannot confirm the booking, the system informs the customer and initiates a refund.

Outcome:
The customer receives tickets that can be printed or displayed on a device. The purchased seats are no longer available for that specific showtime and location. If the customer is signed in, the purchase also appears in their account's purchase history.



3.3.2 Purchase Concessions

Purpose:

<Showtime Hub>

This use case explains how a customer purchases concessions, which include food, drinks, candy, etc, through ShowTime Hub for pickup at a participating theater.

Actor(s) :
The actors are the customer (guest or signed in), the external payment service, participating theater system, and theater staff.

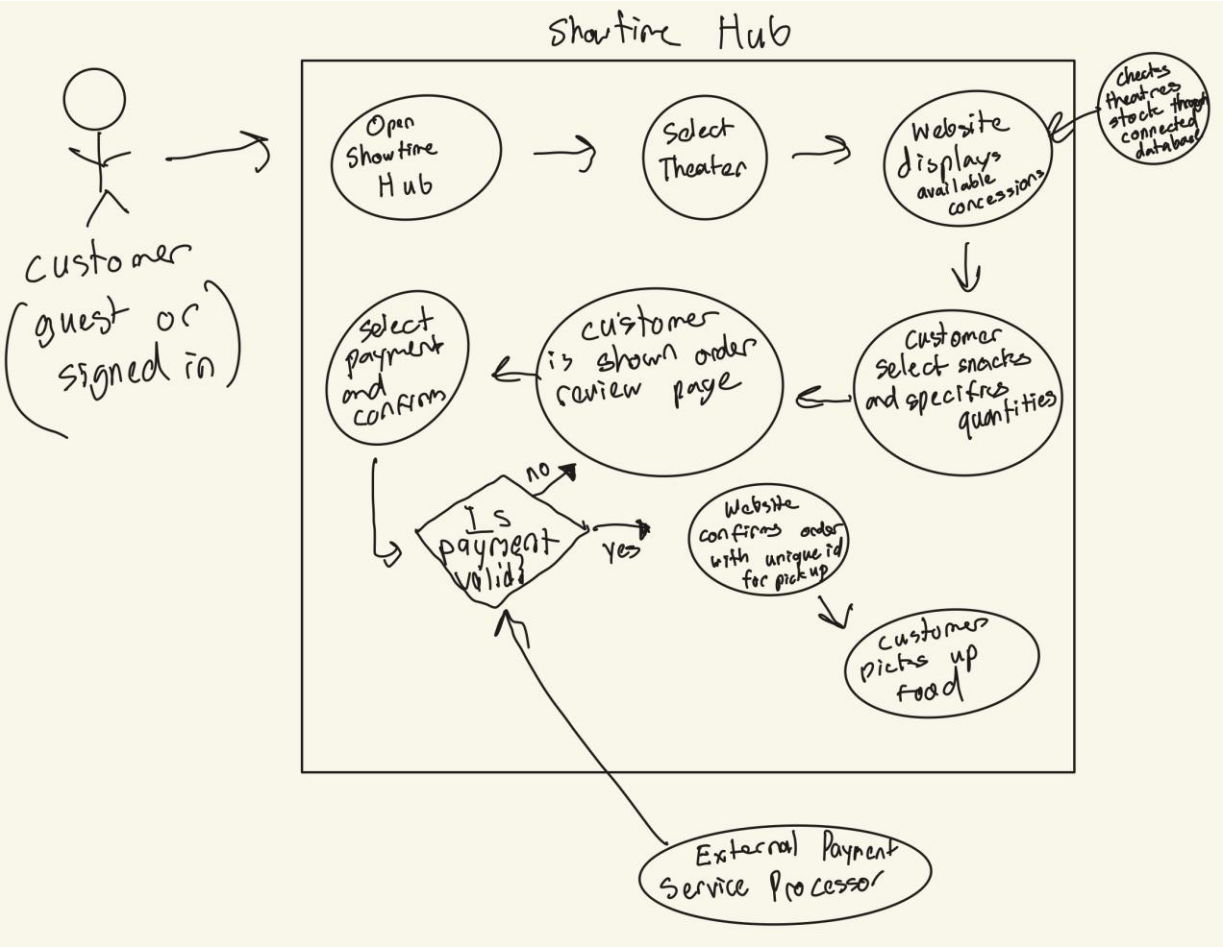
Preconditions:
- The customer has an internet connection
- The customer can access the website.
- The selected theater supports concession purchases through ShowTime Hub.
- Customers don't need an account to place an order

Flow Events:
1. The customer opens ShowTime Hub and selects a participating theater.
2. The system checks the theater's concession availability and displays the availability of those concessions with their prices.
3. The customer selects the items they want and specifies the quantities for each selected item.
4. The system checks that the requested quantities are available and displays an order review page with the selected theater, order, and total price, including applicable taxes and fees.
5. The customer reviews the order and continues to payment.
6. The customer enters their payment info. Signed-in customers can also select a saved payment method if they've saved their payment method before.
7. The customer confirms the purchase, and the system sends the payment request to the external payment service.
8. Once payment goes through and the theater confirms the order, the system records the purchase and displays an order confirmation with a unique claim code.
9. The customer shows the claim code to the theater staff at the selected location to collect the order.

Alternative and Error Flows:
- If an item or requested quantity is not available, the system tells the customer and allows them to change their selection.
- If payment is declined, the system returns the customer to the order review page and allows another payment attempt with a different payment method or allows the user to change their order. Items remain in the cart, but availability is checked again before retrying.
- If the payment result cannot be confirmed, the system shows the order as pending while checking its status.
- If the payment succeeds but the theater cannot confirm the order, the system lets the customer know and causes a payment reversal or refund.
- If the customer cancels before submitting payment, the purchase is not completed, and no claim code is issued.

<Showtime Hub>

Outcome:
- The customer receives confirmation of the concession order and a unique claim code for pickup at the selected theater. If the customer is signed in, the purchase also appears in their account's purchase history.



…

3.3.3: Browse Movie Information

Purpose:
This use case explains how a customer browses current and upcoming movies and views information about a selected movie through Showtime Hub.

Actor(s):

<Showtime Hub>

The actors are the customer (guest or signed in), the participating theater system, and any external movie-information service used to provide movie details.

Preconditions:

- The customer has an internet connection
- The customer can access the ShowTime Hub website.
- Movie listings are available through ShowTime Hub or its connected information sources.
- Customers don't need an account to browse movies

Flow Events:

1. The customer opens the ShowTime Hub.
2. The system displays current and upcoming movies.
3. The customer chooses either "Now Showing" or "Coming Soon".
4. The customer may enter a ZIP code or select a theater location.
5. The system retrieves movies matching the selected category and location. If no location is selected, the results aren't filtered by location.
6. The system displays matching movies with their titles and posters.
7. The customer selects a movie.
8. The system retrieves the selected movie details from its stored information or connected theater and movie-information services.
9. The system displays the movie's title, poster, content rating, synopsis, runtime, genre, director, and cast, along with the available critic scores, user reviews, trailers, and showtimes.
10. The customer may watch the trailer, read reviews, or select an available showtime to go on with purchasing tickets.

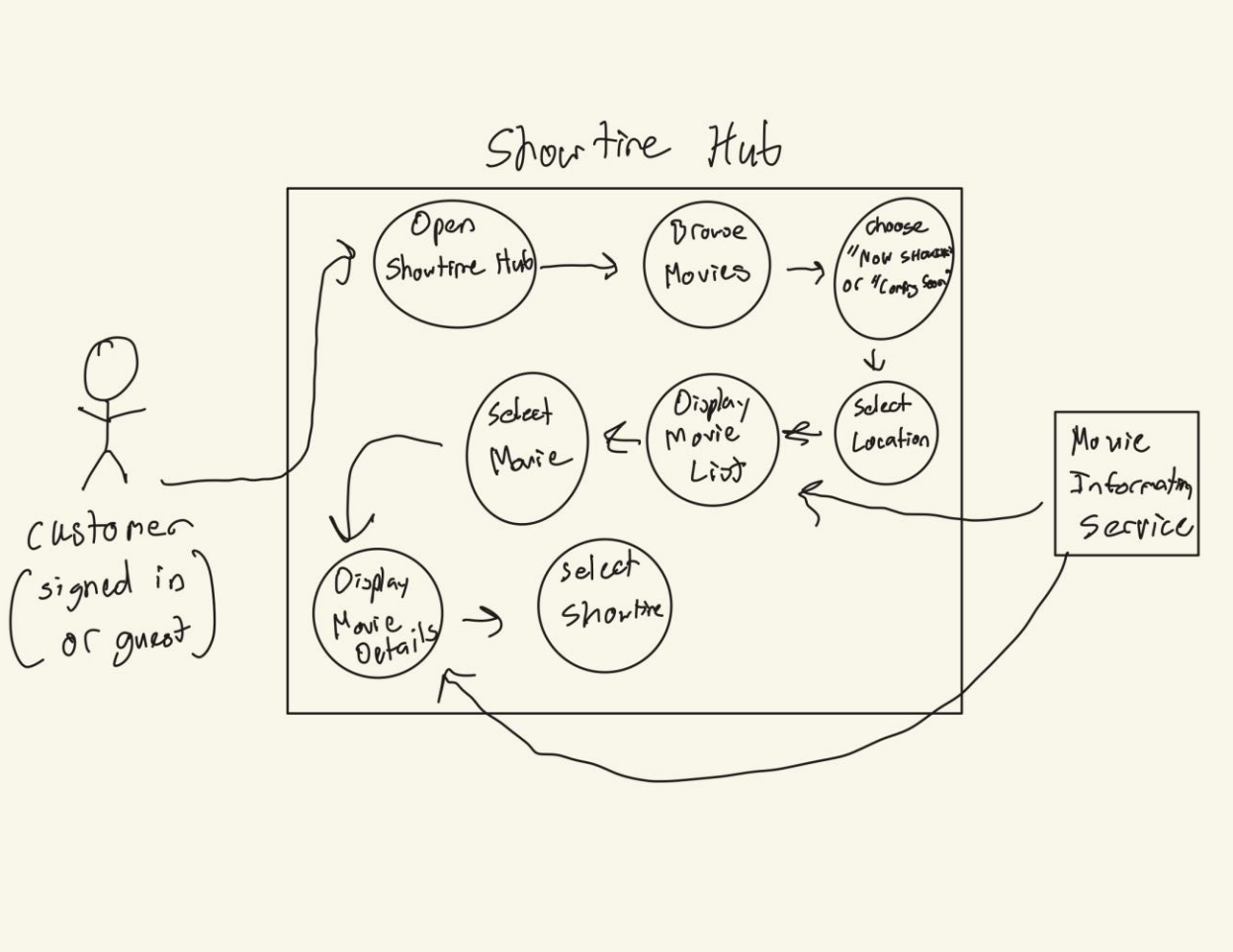
Alternative and Error Flows:

- If no movies match the selected category or location, the system tells the customer and allows them to change their selection.
- If the ZIP code or theater location is not recognized, the system asks the user to try again.
- If movie information cannot be obtained, the system displays an error message and allows the customer to do it again.
- If some information is missing, the remaining movie information is still on display.
- If no showtimes are available for the selected movie and location, the system lets the customer know.
- If the customer leaves the movie details page, they can return to browsing without making a purchase.

Outcome:

- The customer views the selected movie's information and may continue to choose a theater and showtime. No seats are reserved, and no payment is processed during the use case.

<Showtime Hub>



3.4 Classes / Objects

Core classes and objects for ShowTime Hub, are taken from user inputs and the concept of the system described in Section 2.

3.4.1 Class Customer

Customer
Attributes customerID : String name : String email : String passwordHash : String SavedPaymentMethods : List<PaymentMethod>
Functions register() login() viewPurchasedHistory()
Represents a registered customer account. Guests can purchase without creating an account.

3.4.2 Class Payment Method

PaymentMethod
Attributes paymentMethodID : String customerID : String lastFourDigits : String cardType : String expirationDate : String
Functions addPaymentMethod() removePaymentMethod()
This represents payment methods saved to an account. ExpirationDate uses MM/YYYY.

…

3.5 Non-Functional Requirements

3.5.1 Performance

With normal traffic, at least 95% of pages and movie/showtime searches should load within 3 seconds. Seat availability should be updated within 3 seconds after the system receives new information from the theater service.

3.5.2 Reliability

The system will not issue any tickets or codes for concessions until external service confirms that payment has been successful. Seats already sold will not be sold to other users. Failed transactions must not produce confirmed purchases or valid tickets/claim codes, but failure logs are kept for troubleshooting.

3.5.3 Availability

The website will be available at least 99% of each calendar month, excluding scheduled maintenance for system updates. Scheduled updates shall occur during low traffic periods.

3.5.4 Security

All account and payment method related information will be transmitted through secure connections. User passwords and personal data will be stored securely and not as plain text. Passwords will be securely hashed. Admin features are only accessible to authorized admin accounts.

3.5.5 Maintainability

The system should be created and organized such that changes to one major feature like theater integration or payment service provider, do not require major changes to unrelated parts of the website. Any errors should be logged with enough information to identify and fix the issues.

3.5.6 Portability

The website will support current versions of major web browsers, including Chrome, Edge, Firefox, and Safari. The interface should work on both desktop and mobile screen sizes without removing important text and functionality.

4. Software Design Specification

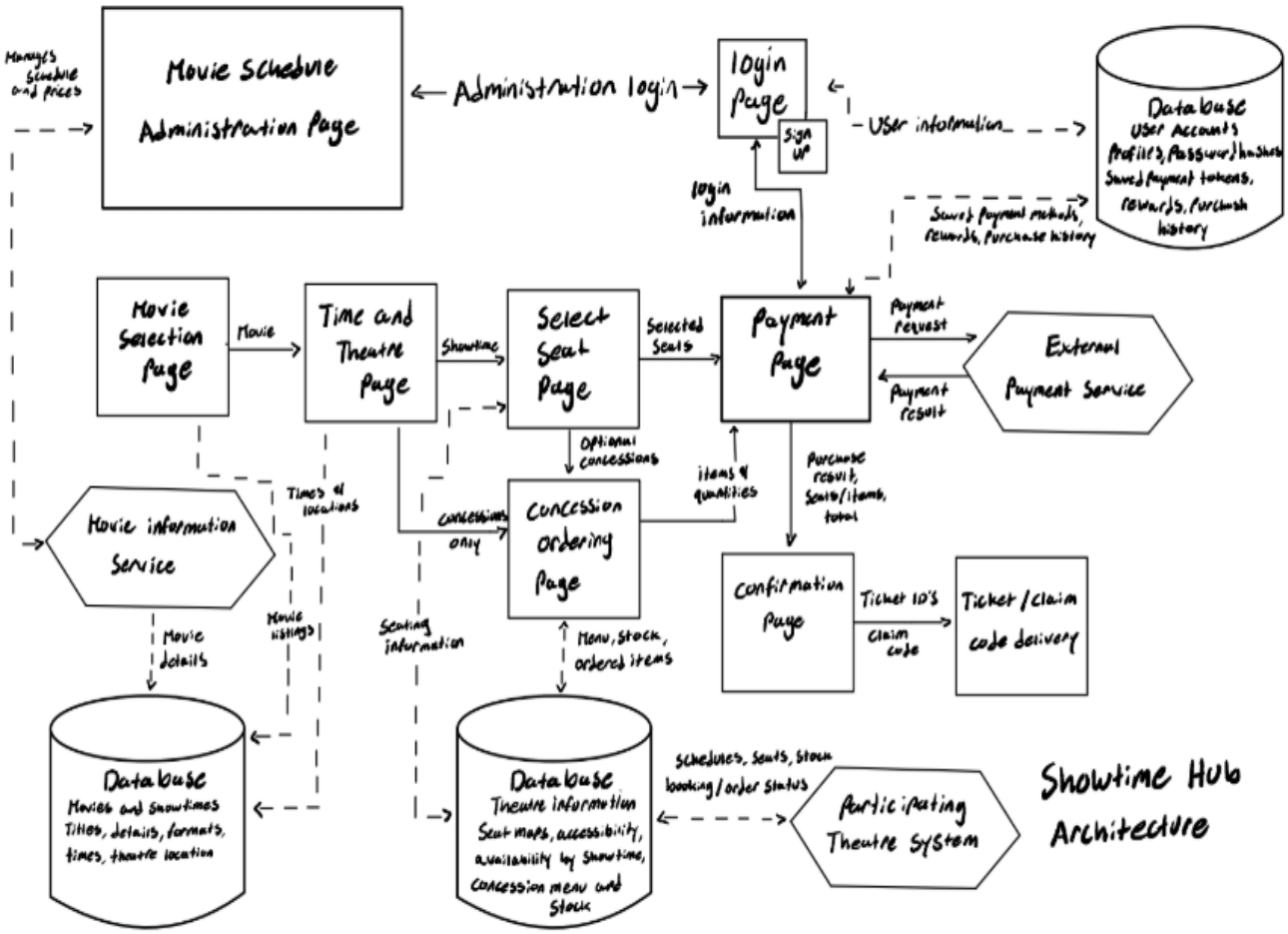
4.1 System Description

Showtime Hub is a web-based movie theater system that allows customers to browse movies and showtimes, select seats, purchase tickets, and order concessions. The system communicates with participating theater systems and an external payment service. Registered customers may also save payment methods and view their purchase history.

4.2 Software Architecture Overview

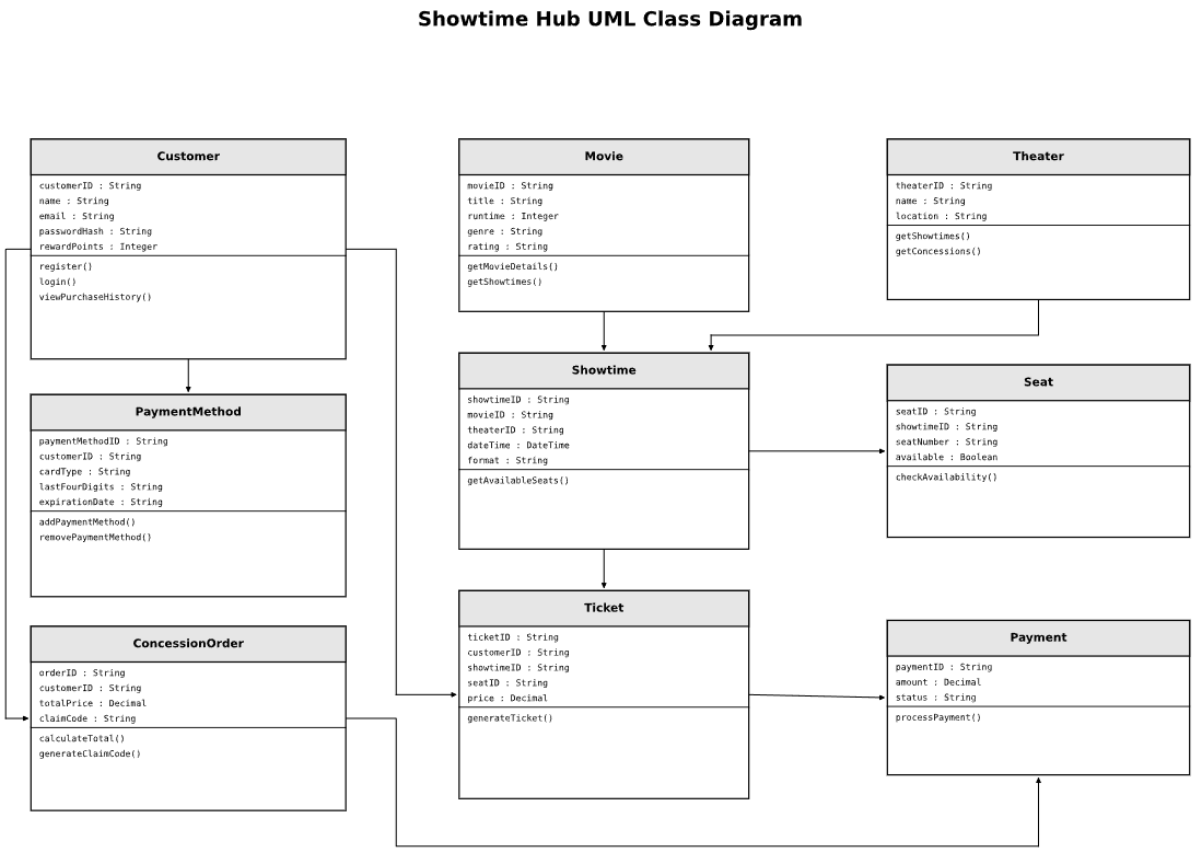
Showtime Hub is divided into several major components. The web interface allows customers and administrators to interact with the system. The application handles movie information, showtimes, ticket purchasing, seat selection, concessions, customer accounts, and payments. The system also communicates with a database, participating in theater systems, and an external payment processor.

4.2.1 Architectural Diagram



4.2.2 UML Class Diagram

<Showtime Hub>



4.3 Class Descriptions

For each major class shown in the UML diagram, include its purpose, attributes, datatypes, and major operations.

4.3.1 Customer

Description: Represents a registered customer account in ShowTime Hub. Customers are able to sign in, save payment methods, view purchases, and accumulate reward points. Guest purchases do not require a Customer account, but these purchases won't be allocated to the reward points.

Attributes:

Attribute	Datatype	Description
customerID	String	Unique identifier for thecustomer
name	String	Customer's name.
email	String	Unique email address used to sign in

passwordHash	String	Securely hashed password used for password authentication.
rewardPoints	Integer	Customer's current reward-point balance.

Operations:

- register(name : String, email : String, password : String) : Boolean
  - This validates the supplied information, checks whether the email is already registered, hashes the pass, and creates the account. Returns true when registration succeeds.
- login(email: String, password : String) : Boolean
  - Checks the supplied credentials against the stored account information. Returns true when authentication succeeds.
- viewPurchaseHistory() : List<Ticket>
  - Returns the customer's previous ticket orders

Relationships: A customer can have multiple PaymentMethods, Ticket, and ConcessionOrders.

4.3.2 PaymentMethod

Description: Represents a payment method saved to a registered customer's account. Showtime Hub stores a payment-service token and limited card information rather than the full card number or security code.

Attributes:

Attributes	**Datatype**	**Description**
paymentMethodID	String	Unique identifier for thesaved payment method.
customerID	String	Identifier of the customer who owns the method.
cardType	String	Card brand or type
lastFourDigits	String	The last four digits that are displayed to help thecustomer identify the card.
expirationDate	String	Card expiration date that is in MM/YYYY format.

<Showtime Hub>

Operations:

- addPaymentMethod (paymentMethod : PaymentMethod) : Void
    - Associates a saved payment method with the customer's account.
- removePaymentMethod(paymentMethodID : String) : Boolean
    - Removes the specified payment method if it belongs to the customer. Returns true when removal succeeds.

Relationships:

Each PaymentMethod belongs to one customer. A saved method may be used for multiple payments.

4.3.3 Movie

Description: Represents a movie listed on Showtime Hub. It provides the info customers view when browsing current and upcoming movies.

Attributes

Attributes	**Datatype**	**Description**
movieID	String	Unique identifier for themovie.
title	String	Movie Title.
runtime	Integer	Movie duration in minutes.
genre	String	Movie category, such as comedy or action.
rating	String	Movie Content Rating

Operations:

- GetMovieDetails() : Movie
    - Returns the movie's information for display on its details page.
- GetShowtimes () : List <Showtime>
    - Returns the movie's scheduled screenings at the specific theater.

Relationships: A Movie can have multiple Showtimes at different theaters.

4.3.4 Theater

Description: Represents a participating theater location. It groups the screenings, seats, and concessions available at that location.

<Showtime Hub>

Attributes

Attribute	Datatype	Description
theaterID	String	Unique identifier for thetheater location.
name	String	Theater name.
location	String	Theater's physical address.

Operations:

- GetShowtime() : List<Showtime>
  - Returns screening scheduled on the supplied calendar date at the theater.
- getAvailableConcessions(): List
  - Returns the theater’s concession menu and current item availability.

Relationships: One theater can have multiple showtimes.

4.3.5 Showtime

Description: Represents a scheduled showing of a movie.

Attributes:

Attributes	Datatype	Description
showtimeID	String	Unique identifier for thescreening.
movieID	String	Identifier of the movie being shown.
theaterID	String	Identifier of the theater hosting the screening.
dateTime	DateTime	The date and starting time of the show.
format	String	The format of the showing, such as IMAX, etc.

Operations:

- getAvailableSeats() : List<Seat>
  - Returns seats currently available for this screening.

Relationships:

Each Showtime belongs to one movie and one theater. Multiple Ticket may refer to the same Showtime.

4.3.6 Seat

Description: Represents a physical seat or designates seating space in a theater auditorium. Its availability depends on the selected Showtime.

Attributes:

Attribute	Datatype	Description
seatID	String	Unique identifier for theseat.
showtimeID	String	The identifier of theshowtime connected to theseat.
seatNumber	String	The displayed seat number, such as A5.
available	Boolean	Shows whether the seat is currently available. True means it is available, and false means the seat has already been selected or purchased.

Operations:

- checkAvailability() : Boolean
    - Checks whether the seat is still available. It returns true if the seat can be selected.

Relationships:

Each seat record belongs to one showtime. One showtime can have multiple seats.

4.3.7 Ticket

Description : Represents a customer's purchase of one or more tickets for a single Showtime. It connects the customer or guest to a showtime and selected seat.

Attributes:

Attribute	Datatype	Description
ticketID	String	A unique identifier for theticket.

customerID	String	The identifier of theregistered customer who purchased the ticket. This value may be empty for a guest purchase.
showtimeID	String	The identifier of a selected showtime.
seatID	String	The identifier of theselected seat.
price	Decimal	The price paid for theticket.

Operations:

- generateTicket(): Ticket
    - Creates and returns the ticket after the purchase is completed

Relationships:

Each ticket is connected to one showtime and one seat; A ticket may also be connected to a registered customer. The ticket purchase is completed through payment.

4.3.8 ConcessionOrder

Description:

The ConcessionOrder class represents an order for food or drinks from a theater. The order receives a claim code that the customer can use when picking it up.

Attributes:

Attribute	**Datatype**	**Description**
orderID	String	Unique identifier for theconcession order
customerID	String	The identifier of thecustomer who placed theorder. This value may be empty for a guest purchase.
totalPrice	Decimal	The total price of all items in the order
claimCode	String	A code used by thecustomer to claim theconcession order.use

Operations:

- calculateTotal() : Decimal
    - Adds the prices of the concession items and returns the total cost.
- generateClaimCode() : String
    - Creates and returns a claim code for the completed order.

Relationships:

A concession order may belong to a registered or guest customer. The order is completed through payment.

4.3.9 Payment

Description: The Payment class represents a payment processed through the external payment service. It keeps track of the amount and whether the payment was successful.

Attributes:

Attribute	Datatype	Description
paymentID	String	A unique identifier for thepayment
amount	Decimal	The total amount being charged
status	String	The current payment status, such as pending, approvedor failed.

Operations:

- processPayment(amount : Decimal) : Boolean
    - Sends the payment amount to the external payment service. It returns true if the payment is approved and false if the payment fails.

Relationships:

A payment can be used to complete a ticket purchase or a concession order.

4.4 Development Plan and Timeline

Development work will be divided among the three group members. Each member will be responsible for specific parts of the Showtime Hub system. All members will also assist with testing, integration, and documentation.

4.4.1 Team Responsibilities

Ivan Rodriguez: Responsible for helping define the software architecture, system components, and overall organization of Showtime Hub

Sang Duong: Responsible for helping develop the UML class diagram, class relationships, and class descriptions.

Diyor Sattarov: Responsible for helping organize the software design specifications, development plan, and ensuring the design remains consistent with the SRS.

4.4.2 Development Timeline

Phase 1 – Design:

Complete the software architecture, UML class diagram, and class descriptions.

Phase 2 – Core Development:

Define how the main features including movie browsing, showtimes, customer accounts, ticket purchasing, seat selection, and concession/snack ordering would be implemented.

Phase 3 – Integration Planning:

Define how Showtime Hub communicates with the database, participating theater systems, movie information services, and external payment services.

Phase 4 – Testing Planning:

Identify how the major system features would be tested, including expected behavior, errors, and interactions between components.

Phase 5 – Finalization:

Review the completed software design, make any necessary corrections, complete the documentation, and prepare the project for submission.

5. Test Plan

5.1 Purpose and Scope

This test plan describes how ShowTime Hub will be tested to verify its requirements and validate its main customer workflows. Testing will focus on movie browsing, ticket generation, seat availability, payment processing, concession price calculations, and concession ordering.

Verification will compare system behavior with the requirements in Section 3 and the class operations described in Section 4. Validation will evaluate whether customers can complete their intended tasks, including browsing movies as a guest, purchasing tickets, and ordering food for pickup. Our Excel sheet includes 10 test cases : four unit tests, three functional tests, and three system tests.

5.2 Testing Setup

Testing will use a test version of the website and connected databases which would contain a small sample of available movies, theaters, showtimes, seats, and food items. Test payment service will be simulated without actually having to enter real credentials and can be manually approved or declined.

Test data will include:

- Movies with titles, ratings, duration, genres, and showtimes
- Seats marked as available or unavailable for a specific showtime
- Concession items with known prices and quantities

Before each test, we will make sure that the required data is available. For example, a successful ticket purchase will need an available seat and response that the booking is confirmed. A declined payment will need the system to decline it.

All data will be reset when switching to different tests to make sure changes don't affect results for other tests.

5.3 Testing Methods

5.3.1 Unit Testing

The unit test plan is going to focus on the Ticket, ConcessionOrder, Seat, and Payment classes described in Section 4.3. We will test their methods separately using already decided inputs. External services will be simulated so that we can control their responses.

Unit_Ticket_1 - Ticket Creation / Generation:

Call generateTicket() using a completed purchase with known showtime and seatIDs. Check that the returned Ticket objects contain the correct information based on what was inputted. Generate another ticket for a different purchased seat and compare the ticket IDs. Both IDs should be properly displayed and different from one another.

Unit_Concession_2 - Concession Purchase Total:

Call calculateTotal() with three items priced at 3.00 each and one item priced at 5.00. The result we should see is $14.00 as a Decimal value. Taxes, fees, and potential discounts will be set to 0 for this testing purpose.

Unit_Seat_3 - Seat Availability:

Call checkAvailability() on an available seat record and an unavailable seat record. We will create one seat record marked available and then another marked unavailable. Each record will belong to a specific showtime. Calling the availability method should return true for the available seat and false for the unavailable seat.

Unit_Payment_4 – Payment Processing:

We will call processPayment() with an amount of $30.00. First, the simulated payment service will approve the payment. Then, in a separate run, it will decline the payment. We can mark a certain sequence of numbers as an accepted payment method. When it is correct, the system will return true. When that diverges, the system should return false for a decline.

5.3.2 Functional and Integration Testing

These tests check feature behavior and the interactions between ticketing, seat availability, payment processing, and participating theater systems. We will compare the confirmation shown on the website with the corresponding ticket, seat, and order records.

Functional_Ticket_1 – Updating Seats After Purchase:

We will select an available seat and complete a ticket purchase with approved payment and theater confirmation. After the purchase, we will check the seat map again. The purchased seat should be unavailable for that showtime, while the other seats should stay unchanged.

Functional_Payment_2 – Declined Payment:

We will attempt to purchase a ticket using a payment that will be decline. The website should display a payment-decline message. It should not confirm the purchase, issue a valid ticket, or mark the seat as sold because of the failed attempt. If the customer retries, the system should check seat availability again. Failure logs can still be saved.

Functional_Concession_3 – Concession Confirmation:

We will select a theater and order available concession items with known prices and quantities. After payment is approved and the theater confirms the order, the website should display the correct order details and a claim code. The total and selected theater should match the customer's choices.

5.3.3 System Tests

System tests will check complete customer tasks through the website. These tests will use the interface, application logic, database, and test services connections together.

System_Ticket_1 – Complete Ticket Purchase:

We will go through the full process of choosing a movie, showtime, and seat. We will review the total and submit the payment. After payment and booking confirmation, the customer should receive an electronic ticket with the correct information. The purchased seat should also become unavailable.

System_Browse_2 – Browsing Without Signing In:

Open the website while signing out and select a currently showing movie. The customer should be able to view the title, rating, runtime, genre, and available showtimes without creating an account. The displayed information should match the test data. Browsing should not reserve a seat or process a payment.

System_Concession_3 – Concession Order and Pickup:

We will select a theater, add concession items, review the total, and complete payment. Once the order is confirmed, the customer should receive a claim code. Theater staff should be able to use the code to find the matching paid order and provide the correct items.

5.4 Test Cases and Connections to SRS

The tests connect back to the other sections in our SRS. This table just displays how they are connected and what we have covered:

Test Case	SRS Connection	Main Class or Feature
Unit_Ticket_1	3.2.1.4 and 4.3.7	Ticket
Unit_Concession_2	3.2.2.3 and 4.3.8	ConcessionOrder
Unit_Seat_3	3.2.1.3 and 4.3.6	Seat
Unit_Payment_4	3.2.1.3 - 3.2.1.5 and 4.3.9	Payment
Functional_Ticket_1	3.2.1 and 3.3.1	Ticket purchasing
Functional_Payment_2	3.2.1.5 and 3.5.2	Declined ticket payment
Functional_Concession_3	3.2.2 and 3.3.2	Concession ordering
System_Ticket_1	3.3.1	Complete ticket purchase
System_Browse_2	3.2.3 and 3.3.3	Guest movie browsing
System_Concession_3	3.3.2	Concession order and pickup

5.5 Errors and Limits of the Tests

Our test cases check both successful actions and some failure conditions. These include declined payments, unavailable seat responses, incorrect totals, and ticket information that doesn't match the purchase.

The 10 samples don't cover every possible problem. Additional tests would be for situations such as 2 customers buying the same seat at the same time, concessions going out of stock, a payment staying pending, or a theater failing to confirm an order after payment.

We would also need separate tests for performance, security, accessibility, and browser compatibility. Passing these 10 cases would show that the tested examples worked but wouldn't prove that every feature in the SRS works correctly.

5.6 Recording Test Results

Each test will follow the prerequisites and steps listed in the Excel document. We will compare what happens with the expected result and record the actual result.

The Status column will use:

- Not Run: The test has not been performed.
- Pass: All expected results were observed.
- Fail: One or more expected results were not observed.
- Blocked: The test could not be completed because something required was unavailable.

(PS: We only used to "Not Run", as we are not actually testing the website.)

For tests with more than one scenario, such as approved and declined payments, we will record both results. Both scenarios must work for the whole test to pass.

5.7 Test Documents and GitHub Links

The test plan is included in the SRS, and the detailed test cases are stored in the Excel workbook. The GitHub folder will contain both documents.

- GitHub Link: https://github.com/Kyo525/ShowTime-Hub/tree/main
- Excel Test Cases: https://docs.google.com/spreadsheets/d/1MaEUjrK7sV3jn3LZSSPchsDWozjdbViB/edit?usp=sharing&ouid=100027373225209407662&rtpof=true&sd=true